Graph-RAG Knowledge Dashboard (Project 576)

A Hybrid Vector–Graph Retrieval Engine with Universal Cognitive Force-Dynamic Primitives: Architecture, Engineering History, and Cross-System Evaluation

Project Summary & Final Report
University of Waikato | October 2026

Abstract

Retrieval-Augmented Generation (RAG) systems built exclusively on dense vector databases frequently fail at *multi-hop reasoning*—synthesizing logical and causal chains whose implied facts span semantically-disjoint document sections—and remain vulnerable to ungrounded hallucinations when direct links are absent. Conversely, contemporary Knowledge Graph RAG (Graph-RAG) frameworks suffer from a fundamental schema trade-off: unconstrained Open Information Extraction (Open IE) incurs high token overhead and predicate entropy, whereas rigid domain-specific Web Ontology Language (OWL) schemas, such as those of TrustGraphAI, reject valid domain facts and collapse under topic drift. This report presents the design, 10-week engineering lifecycle, full-stack implementation, and empirical validation of the **Graph-RAG Knowledge Dashboard (Project 576)**. This system introduces the **Talmy Regime**: a hybrid dual-state-space retrieval engine combining 768-dimensional Matryoshka vector embeddings in PostgreSQL (`pgvector`) with a Neo4j property graph governed by Leonard Talmy's 7 closed-class semantic Force Dynamics primitives (Θ), enriched with unconstrained per-edge mechanism descriptions. The complete full-stack web application is detailed (FastAPI backend + React/Vite interactive dashboard), its zero-token idempotent cache synchronization protocol, multi-tier error handling, security hardening, our three-stage evaluation benchmark against an adapted `trustgraph-fork` baseline across 108 evaluation runs and an out-of-domain stress test, and the condensed out-of-the-box execution guide.

Contents

1 Project Description and Production Vision **2**
1.1 The Multi-Hop Reasoning Problem in Enterprise RAG . . . 2
1.2 How the Finished System is Used in Practice . . . 2
1.3 Architectural Innovations vs. Contemporary Graph-RAG Systems . . . 3

2 System Architecture, Technologies Used, and Formal Retrieval Model **4**
2.1 Full-Stack Technology Stack and Dual State-Space Topology . . . 4
2.2 Formal Five-Stage Hybrid Retrieval Operator . . . 4

3 Sequence of Steps: End-to-End User Application Walkthrough **6**

4 Completed Functionality vs. Planned / Unfinished Functionality **8**
4.1 Completed Functionality (100% Operational) . . . 8
4.2 Planned Functionality Not Finished or Architecturally Pivoted . . . 9

5 Multi-Level Error Handling and System Resilience **10**

6 Testing, Security Verification, Bug Resolution, and Benchmarking **11**
6.1 Automated Testing, Security Verification, and Bug Resolution Ledger . . . 11
6.2 Three-Stage Comparative Benchmark: Project 576 vs. TrustGraph . . . 12

7 Engineering Challenges Encountered and How They Were Overcome **14**

8 Condensed Codebase Architecture and Out-of-the-Box Execution Guide **17**

8.1 Three-Step Local Execution Workflow 17

9 AI Usage Notes **18**

1 Project Description and Production Vision

1.1 The Multi-Hop Reasoning Problem in Enterprise RAG

Retrieval-Augmented Generation (RAG) grants Large Language Models (LLMs) access to private or rapidly evolving document bundles that were not present in their pre-training weights [1]. Standard industrial RAG pipelines rely primarily on **Vector Databases**, which project text chunks into high-dimensional coordinate spaces ($\mathbb{R}^d$) and retrieve passages via cosine or inner-product similarity against an embedded user query. Conceptually, dense vector search acts as a semantic generalization of `grep`: it locates passages that are lexically or conceptually proximate to the prompt.

Even though vector retrieval excels at single-fact lookups, it breaks down in dense-thinking workflows where the answer requires **multi-hop causal reasoning**. If Chunk A establishes that *"Autoregressive Filtration removes synthetic samples below confidence threshold δ"* and Chunk B (ten pages later) states that *"Filtered synthetic data regularizes Model C against sharp loss minima,"* a query asking how threshold δ affects loss-landscape geometry must bridge two vectorially distant passages through an intermediate entity (*Filtered Synthetic Data*). When dense retrieval misses one link in the chain, the LLM either guesses or hallucinates plausible-sounding mechanisms.

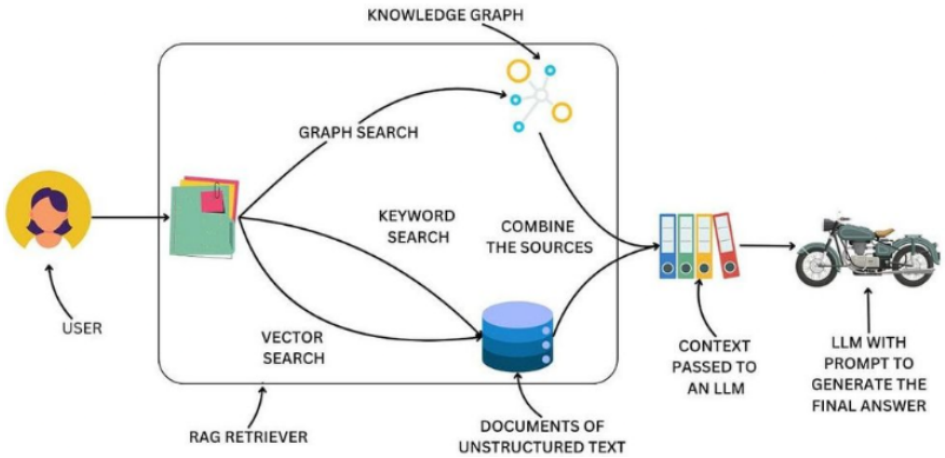


Figure 1: Conceptual Hybrid Graph-RAG retrieval architecture integrating keyword indexing, continuous vector semantics, and discrete knowledge graph traversal [6].

To overcome this limitation, this project engineers a full-stack **Hybrid Graph-RAG Knowledge Dashboard (Project 576)** that unifies continuous vector similarity search ($\mathcal{S}_{\text{cont}}$) with discrete topological graph traversal ($\mathcal{S}_{\text{disc}}$), as illustrated in Figure 1. Every synthesized claim is anchored to a deterministic provenance chain:

$$\text{Query } q \longrightarrow \text{Seed Chunks } S_q \longrightarrow \text{Seed Entities } V_0 \longrightarrow k\text{-Hop Subgraph } E^* \longrightarrow \text{Grounded Answer } A_d(q)$$

1.2 How the Finished System is Used in Practice

In its completed form, the Graph-RAG Knowledge Dashboard serves as an auditable, hallucination-fortified analytical workstation for domain professionals who cannot accept opaque "black-box" AI

summaries:

1. **Regulatory Compliance & Legal Auditing:** Legal analysts upload statutory codes, contracts, or compliance manuals (PDFs). When querying how a specific regulatory clause constrains downstream operational liabilities, the dashboard renders both a prose synthesis and an interactive causal graph showing the exact chain of obligations, citing the verbatim source chunks.
2. **Scientific & Machine Learning Literature Synthesis:** Researchers upload technical papers (such as deep learning architecture reports or pure mathematics texts) and query complex ablation dependencies, hyperparameter interactions, or theorem prerequisites. Users can click inline edge citations—formatted as `(SOURCE) -[RELATION]-> (TARGET)`—to inspect the exact mechanism and highlight the corresponding topological path on the visual canvas.
3. **Software Architecture & Incident Root-Cause Analysis:** Engineering teams ingest post-mortems and system design documents to trace cascading failure modes across microservices without risking context contamination between unrelated incidents.
4. **Zero-Contamination Session Isolation & Reproducible Audit Trails:** Every query is strictly scoped to a cryptographically hashed document identity (SHA-256), preventing cross-document context pollution. Every completed execution is permanently snapshotted in an append-only database ledger (`query_runs`), allowing auditors to replay historical analyses instantaneously at zero LLM token cost.

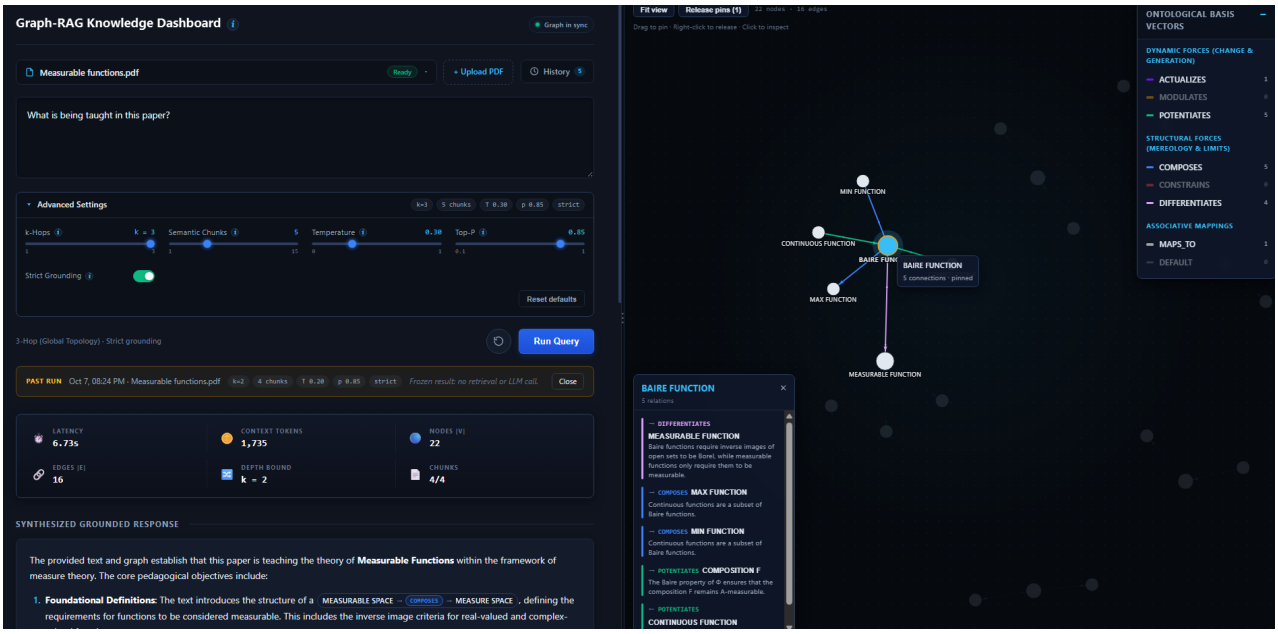


Figure 2: The completed Graph-RAG Knowledge Dashboard (Week 10 production interface). Left pane ($\mathcal{S}_{\text{cont}}$): Document picker, upload progress, query console, Advanced Settings, frozen run replay banner, live telemetry ribbon, and Markdown/LaTeX synthesized response with inline ontology badges. Right pane ($\mathcal{S}_{\text{disc}}$): Interactive D3/Canvas force-directed causal graph, node inspection card, and Talmy Ontological Basis Vectors filter.

1.3 Architectural Innovations vs. Contemporary Graph-RAG Systems

Existing Graph-RAG frameworks exhibit severe architectural trade-offs that motivated our design:

- **Query-Ready k-Hop Traversal vs. Leiden Pre-Clustering:** Microsoft's GraphRAG [2] relies on batch community detection via the Leiden algorithm, incurring $\mathcal{O}(m \log m)$ global pre-processing overhead over m edges and $> 2\times$ query latency. Following the local traversal philosophy

of TrustGraph [5], our engine remains immediately query-ready by anchoring in PostgreSQL (`pgvector`) and executing a bounded k-step Breadth-First Search (BFS) of complexity $\mathcal{O}(d^k)$ (for average degree d and horizon $k \in \{1, 2, 3\}$) directly in Neo4j.

- **The Knowledge Graph Schema Dilemma (Universal Cognitive Primitives):** Structuring a knowledge graph forces a three-way ontological choice:
  (a) *Open Information Extraction (Open IE, e.g., TrustGraph `everything` flow):* Allows unconstrained natural-language predicates ($\mathcal{R} \subseteq \Sigma^*$). While expressive, it suffers from unbounded relational entropy, tautological/redundant edges, and high extraction cost.
  (b) *Domain-Specific OWL Ontologies (e.g., TrustGraph `ontology` / OntoRAG flow):* Enforces strict taxonomic classes and properties ($\mathcal{C}_{\text{domain}}, \mathcal{R}_{\text{domain}}$). As proved in our Stage 3 benchmarks, rigid domain schemas act as over-aggressive filters that discard valid facts (causing catastrophic graph sparsity) and collapse completely under domain shift.
  (c) *The Talmy Regime (Project 576):* We solve this dilemma by grounding edge extraction in cognitive linguist Leonard Talmy's **Force Dynamics** [3]. We restrict graph edges to a closed, domain-invariant basis of **7 universal causal/structural primitives** (Θ), while preserving rich domain nuance inside a free-text `detail` property on every edge and combining graph edges with raw text chunks in a unified **Gestalt Context**.

2 System Architecture, Technologies Used, and Formal Retrieval Model

2.1 Full-Stack Technology Stack and Dual State-Space Topology

Table 1 catalogs the production technologies integrated across Project 576 and the comparative benchmark suite. Knowledge representation is partitioned across two coupled state spaces:

1. **Continuous Semantic State ($\mathcal{S}_{\text{cont}}$ — Supabase PostgreSQL):** Stores document metadata (`documents`), overlapping text chunks (`document_chunks`) embedded into $\mathbb{R}^{768}$ via `gemini-embedding-001`, the per-chunk `extracted_graph` JSONB cache stamped with a 16-char prompt signature (`ontology_hash`), and the append-only `query_runs` ledger.
2. **Discrete Topological State ($\mathcal{S}_{\text{disc}}$ — Neo4j AuraDB):** A deterministic property-graph projection of the Supabase JSONB cache. Entities are unified globally by canonical `UPPER_SNAKE_CASE` name, while causal edges carry per-chunk and per-document provenance:

$$\text{Edge } e = (u, v, \theta, \mathbf{p}), \quad u, v \in V \text{ (nodes)}, \quad \theta \in \Theta \text{ (category)},$$
$$\mathbf{p} = (\delta \in \Sigma^*,\ c \in [0,1],\ \texttt{chunk_id}\ x,\ \texttt{doc_id}\ d)$$

and $\mathbf{p}$ is description, confidence, and provenance.

Table 2 defines the 7 orthogonal basis vectors Θ (`ontologies/hybrid_semantic_partition.py`) and their projection confidence thresholds $\tau(\theta)$ (`pipeline/graph_sync.py`).

2.2 Formal Five-Stage Hybrid Retrieval Operator

Given a ready document $d \in D$ with chunk set X_d and query $q \in \Sigma^*$, `HybridQueryEngine` (`retrieval/query_engine.py`) computes the grounded synthesis $A_d(q)$ in five stages:

1. **Semantic Anchoring (f_{vector}):** Embeds $v_q = \varphi_Q(q) \in \mathbb{R}^{768}$ (`RETRIEVAL_QUERY`) and retrieves top-κ chunks via `pgvector` cosine distance, calibrating UI relevance $\rho(x)$ onto $[0, 1]$:

$$S_q = \text{Top}_\kappa\big(X_d,\ \text{sim}(q, x)\big), \qquad \text{sim}(q, x) = 1 - (v_q \texttt{ <=> } v_x), \qquad \rho(x) = \text{clip}_{[0,1]}\left(\frac{\text{sim}(q, x) - 0.54}{0.80 - 0.54}\right)$$

Table 1: Technologies utilized in Project 576 and the Stage 2–3 Comparative Benchmark Suite.

Subsystem	Technology / Library	Architectural Role in Project
Frontend UI	React 19, Vite, `react-split`	Responsive bifurcated single-page dashboard on `http://localhost:5173` with resizable continuous/discrete panes and `sessionStorage` state persistence.
Graph Visualization	`react-force-graph-2d`, HTML5 Canvas, D3-Force	Interactive force-directed node-link rendering of $G_{\text{local}} = (V_{\text{local}}, E_{\text{local}})$ with degree scaling, node pinning, adjacency highlighting, and ontology filtering.
Answer Rendering	`react-markdown`, `remark-math`, `rehype-katex`, KaTeX	Compiles LLM prose, mathematical LaTeX notation, and inline `(SRC) -[REL]-> (TGT)` citations into interactive, color-coded ontology badges.
Backend API	Python 3.13, FastAPI, Uvicorn, Pydantic v2	Asynchronous REST server (`http://127.0.0.1:8000`) with thread-pool offloading (`asyncio.to_thread`), strict schema validation, and background job orchestration.
Continuous State ($\mathcal{S}_{\text{cont}}$)	Supabase PostgreSQL, `pgvector` (HNSW), `psycopg2`	Relational storage for `documents`, `document_chunks` (768-d cosine HNSW index + `extracted_graph` JSONB cache), `query_runs` ledger, and PDF Blob Storage.
Discrete State ($\mathcal{S}_{\text{disc}}$)	Neo4j AuraDB, Cypher, `neo4j` Bolt Driver	Cloud graph database storing `(:Document)-[:HAS_CHUNK]->(:Chunk)` and `(:Chunk)-[:MENTIONS]->(:Entity)` plus the 7 Talmy force-dynamic edge types.
Neural Models (LLM & Embeddings)	Google GenAI SDK (`gemini-embedding-001`, `gemini-3.1-flash-lite`), Groq API	Asymmetric 768-d Matryoshka embeddings (`RETRIEVAL_DOCUMENT` / `RETRIEVAL_QUERY`), batched JSON-schema graph extraction, intent fallback, and grounded synthesis. (Groq `llama-3.3-70b` used in Weeks 4–5 prototyping).
Benchmark Baseline & Harness	Podman, `socat`, Apache Pulsar, Qdrant, Cassandra, FlashRank, `gemini-3.8-flash`	Stage 2 `trustgraph-fork` microservice deployment with 768-d vector parity, WebSocket PROV-O triple extraction, and Stage 3 NLI LLM-as-a-judge evaluation.

Table 2: The 7 Closed-Class Talmy Force Dynamics Primitives (Θ) and Projection Confidence Thresholds $\tau(\theta)$.

Force Category	Primitive $\theta \in \Theta$	Formal Cognitive / Physical Definition	Threshold $\tau(\theta)$
Dynamic Forces	`ACTUALIZES`	Vector of generation ($0 \to 1$): an active force brings an entity or outcome into existence.	0.80
(Change & Generation)	`MODULATES`	Vector of transformation ($x \to x'$): continuously alters the state, phase, or intensity of an existing entity.	0.80
	`POTENTIATES`	Vector of affordance: removes a barrier or supplies the necessary substrate/conditions for another entity to act.	0.80
Structural Forces	`COMPOSES`	Vector of mereological synthesis: smaller constituent parts bind to form a unified whole (part-to-whole).	0.70
(Mereology & Limits)	`CONSTRAINS`	Vector of restriction: applies pressure to bound, limit, or halt the trajectory/capacity of another entity.	0.70
	`DIFFERENTIATES`	Vector of contrast (*Contrast Rule*): emitted whenever text contrasts A vs. B; `detail` must name the distinguishing attribute.	0.70
Associative	`MAPS_TO`	Vector of structural alignment or measurement correspondence without direct causal energy transfer.	0.55

2. **Bipartite Seed Resolution:** Resolves seed entities mentioned by S_q: $V_0 = M(S_q) = \{v \in V \mid \exists x \in S_q, (x, v) \in M\}$.
3. **Bounded k-Hop Graph Traversal (f_{graph}):** Walks undirected Cypher trails of length $1 \dots k$ ($k \in \{1, 2, 3\}$) scoped to document d's edges $\mathcal{E}_d$, restoring canonical direction and capping at 300 triples:

$$\Gamma_k(V_0) = \big\{e = (s, \theta, t, \dots) \in \mathcal{E}_d \;\big|\; \min\big(\text{dist}_d(V_0, s), \text{dist}_d(V_0, t)\big) \le k - 1\big\}, \qquad E_k = \text{cap}_{300}\big(\Gamma_k(V_0)\big)$$

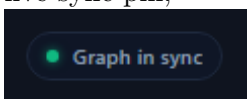
4. **Question Intent Focus ($\iota(q)$):** Regex patterns (`INTENT_PATTERNS`) or a cached 64-token LLM fallback identify up to 2 target primitives $F = \iota(q)$, pulling up to 40 adjacent focus edges E_F and sorting them first: $E^* = \text{sort}_F(E_k \cup E_F)$.
5. **Gestalt Fusion & Synthesis ($f_{\text{fuse}} \circ f_{\text{synth}}$):** Fuses S_q and E^* (with `Mechanism: {detail}`) into prompt $\Psi(q, S_q, E^*, F)$ under strict (σ_{strict}) or open segregated (σ_{open}) grounding:

$$A_d(q) = f_{\text{synth}}\Big(\Psi\big(q,\, S_q,\, \text{sort}_F(\Gamma_k(M(S_q)) \cup E_F),\, \iota(q)\big);\; \sigma_g, T, p\Big)$$

3 Sequence of Steps: End-to-End User Application Walkthrough

An analyst interacting with the dashboard at `http://localhost:5173` executes a streamlined seven-step workflow:

1. **Verify Cross-Database Synchronization (`GraphSyncBadge`):** The header bar displays the live sync pill,



which calls `GET /api/graph/sync` ($\sim$ 250 ms) to verify that every Supabase chunk's `graph_hash` matches Neo4j. Any projection drift can be repaired in zero LLM tokens via **Repair (free)** (L1 heal) or re-extracted via **Re-extract** (L2 heal).

2. **Select or Upload a PDF Document:** Via the top document bar (Figure 3), the user selects a `Ready` document or clicks **+ Upload PDF** (`POST /api/upload`). Identical files resolve immediately via SHA-256 deduplication (`cached: true`); new PDFs stream live progress across `queued` $\rightarrow$ `store` $\rightarrow$ `parse` $\rightarrow$ `embed` $\rightarrow$ `extract` $\rightarrow$ `finalize`.
3. **Configure Query & Advanced Settings:** The user enters a question and optionally opens **Advanced Settings** (Figure 4) to tune k-hops ($k \in \{1, 2, 3\}$), top-κ semantic chunks ($\kappa \in [1, 15]$), temperature $T \in [0.0, 1.0]$, top-$p \in [0.1, 1.0]$, and **Strict Grounding** (σ_{strict} vs. σ_{open}).
4. **Execute Query & Inspect Real-Time Telemetry:** Clicking **Run Query** (`POST /api/query`) populates the **Telemetry Ribbon** with backend **Latency**, **Context Tokens**, subgraph **Nodes** $|V|$, **Edges** $|E|$, **Depth Bound** (k), and **Chunks** returned.
5. **Audit Grounded Answer, Inline Badges, and Source Chunks:** The response pane renders Markdown/KaTeX prose with clickable color-coded Talmy edge pills (Figure 5a) above the **Semantic Anchors** cards showing verbatim text, raw cosine (`cos 0.711`), and calibrated relevance (`66%`, Figure 5b). Out-of-scope queries under σ_{strict} deterministically halt with `INSUFFICIENT_CONTEXT` (Figure 6).
6. **Explore the Force-Directed Causal Graph ($\mathcal{S}_{\text{disc}}$):** In the right pane (Figure 2), hovering isolates adjacent neighborhoods, clicking a node pins it and opens its `Mechanism: {detail}` card, and clicking any of the 7 **Ontological Basis Vectors** filters the canvas by force primitive.
7. **Replay Frozen Sessions (`RunTimeline`):** Clicking **History** opens the append-only ledger drawer (`GET /api/runs/{id}`), rehydrating any past query, graph, and telemetry snapshot in **zero LLM tokens**.



Figure 3: Document context bar showing the active PDF (`Measurable functions.pdf`), readiness status badge, `+ Upload PDF` trigger, and `History` drawer counter.

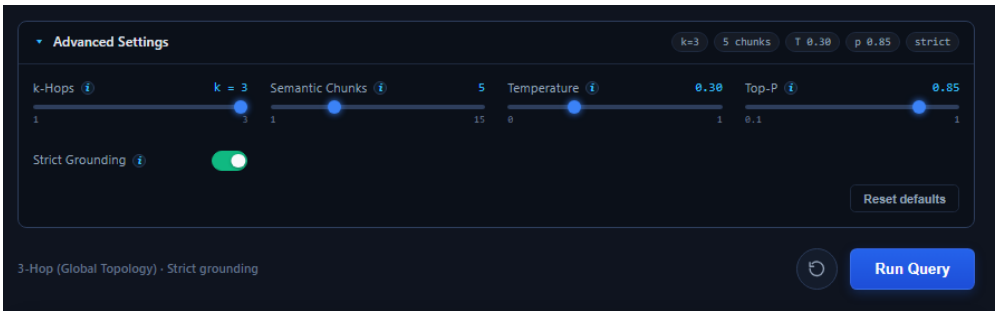
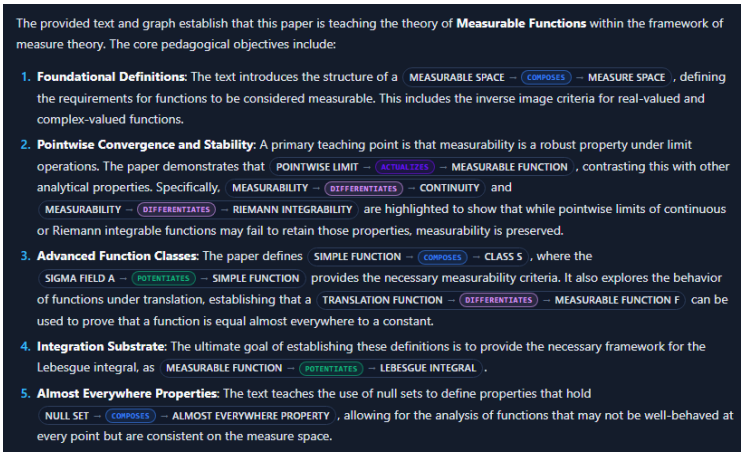


Figure 4: Collapsible Advanced Settings panel exposing k-hops, top-κ semantic chunks, temperature, top-p, and the Strict Grounding toggle.

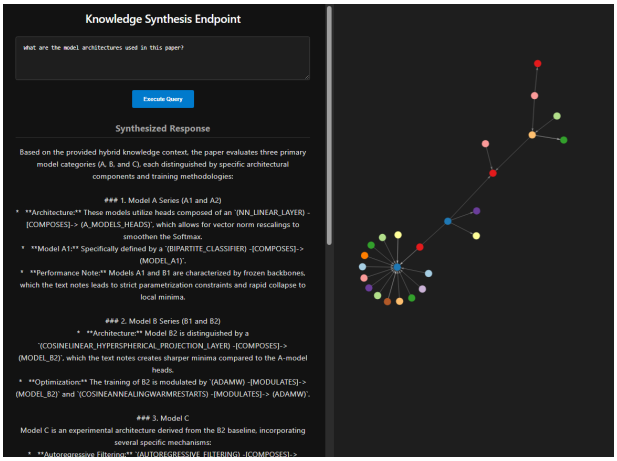


(a) Synthesized Grounded Response with inline color-coded Talmy ontology citation badges.

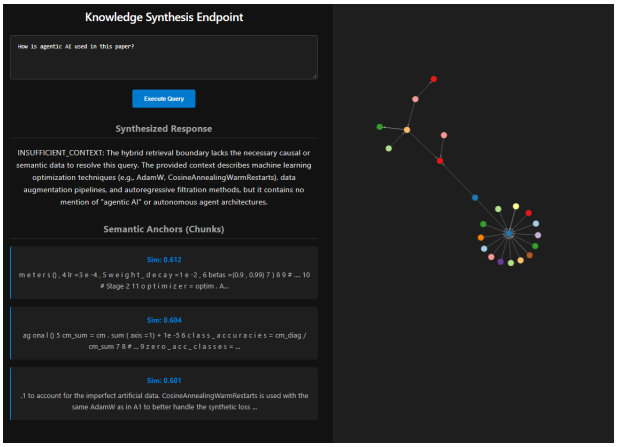


(b) Semantic Anchor chunk cards showing raw cosine similarity and verbatim text.

Figure 5: Continuous-state verification: inline ontological edge citations and retrieved semantic anchor chunks.



(a) Within-bounds query: synthesizes multi-hop model architecture comparisons and renders the local causal subgraph.



(b) Out-of-bounds query (*"How is Agentic AI used?"*): triggers deterministic `INSUFFICIENT_CONTEXT` refusal.

Figure 6: Verification of Constrained Generation under σ_{strict} for in-scope vs. out-of-scope queries.

4 Completed Functionality vs. Planned / Unfinished Functionality

4.1 Completed Functionality (100% Operational)

Table 3 catalogs the completed functional and non-functional subsystems delivered in the final build.

Table 3: Summary of Completed Functionality Across Backend, Database, Frontend, and Evaluation Subsystems.

Feature / Subsystem	Primary Modules	Implemented Capabilities
1. Content-Addressed Multi-Doc Ingestion	`main.py`, `jobs.py`, `ingest.py`	Asynchronous PDF upload (≤ 25 MB), SHA-256 byte deduplication, Supabase Blob Storage backup, and resumable 6-stage background job pipeline.
2. Deterministic Sliding-Window Chunker	`pipeline/ingest.py`	Zero-cost Python chunker (1000 chars, 200 overlap) snapping to natural sentence/word boundaries with infinite-loop protection on unbroken strings.
3. Matryoshka 768-d Vector Projection	`sync_embeddings.py`, `query_engine.py`	Asymmetric `gemini-embedding-001` projections (`RETRIEVAL_DOCUMENT` vs. `RETRIEVAL_QUERY`) truncated to $\mathbb{R}^{768}$ and indexed via `pgvector` HNSW with `iterative_scan = strict_order`.
4. Batched Talmy Graph Extraction	`assemble_graph.py`, `ontologies/`	Concurrent 4-chunk batched Gemini structured JSON extraction enforcing the 7 Talmy Force Dynamics primitives (Θ), confidence scores, and `detail` mechanisms.
5. L0/L1/L2 Cache-First Graph Sync	`graph_sync.py`, `GraphSyncBadge.jsx`	Cryptographic `ontology_hash` and per-chunk `graph_hash` verification; automatic zero-token L1 drift repair from Supabase JSONB to Neo4j and explicit L2 re-extraction.
6. Hybrid Retrieval & Intent Focus	`query_engine.py`	Combines cosine top-κ chunk anchoring, undirected k-hop Cypher BFS ($k \in \{1, 2, 3\}$), regex + cached-LLM intent detection ($\iota(q)$), and up to 40 intent-focused edges.
7. Gestalt Synthesis & Grounding Modes	`query_engine.py`	Fuses source chunks and causal mechanisms; supports strict grounding (σ_{strict} with `INSUFFICIENT_CONTEXT` refusal) and open segregated mode (σ_{open}).
8. Split-Pane Interactive React GUI	`frontend/src/**`	Resizable dual-pane dashboard with Markdown/KaTeX rendering, clickable inline edge badges, calibrated relevance bars, and interactive D3 force graph.
9. Immutable History Replay Ledger	`002_query_runs.sql`, `RunTimeline.jsx`	Append-only `query_runs` table (protected by an `UPDATE`-blocking SQL trigger) enabling zero-token instant replay of historical queries across sessions.

Table 3 (continued)

Feature / Subsystem	Primary Modules	Implemented Capabilities
10. Three-Stage Benchmark Suite	`evaluation-harness`, `trustgraph-fork`	End-to-end comparative harness evaluating Project 576 against Open IE and Domain OWL regimes across 5 formal metrics ($M_{\text{corr}}, M_\Phi, M_\Sigma, \eta, M_T$).

4.2 Planned Functionality Not Finished or Architecturally Pivoted

In accordance with the report requirements, Table 4 details functionality originally envisioned in the proposal or intermediate weekly plans that was either partially completed, architecturally pivoted, or left unstarted, along with its exact completion state and engineering rationale.

Table 4: Planned Functionality Not Fully Completed or Pivoted, Completion State, and Wider Architectural Context.

Planned Feature	Progress	Current State & Context for Wider Design Decisions
1. Docker Containerization & Render Cloud Deployment of Project 576 *(Proposal Sec. 7)*	**45%** *(Pivoted)*	**State:** Both databases (Supabase PostgreSQL and Neo4j AuraDB) are 100% cloud-hosted. However, containerizing the Project 576 FastAPI/React app for Render was halted in Week 8 due to Windows host hypervisor virtualization restrictions. Furthermore, Trustgraph uses containerization primarily because it can run its databases entirely locally, something this project did not follow. **Wider Context:** Pivoted containerization efforts to Stage 2, where an 18-container Podman stack was orchestrated with custom `Dockerfile.embeddings` and `socat` network bridges for `trustgraph-fork`, while running Project 576 natively on localhost for rapid algorithmic iteration.
2. Cross-Document Multi-Hop Graph Traversal *(Stage 1 / Week 10)*	**70%** *(Scoped by Design)*	**State:** The Neo4j storage layer already merges `(:Entity {name})` globally across all uploaded PDFs, meaning shared concepts physically link multiple documents in $\mathcal{S}_{\text{disc}}$. However, the query-time Cypher walk enforces `WHERE rel.doc_id = $doc_id`. **Wider Context:** Allowing unconstrained cross-document hops caused context contamination between contradictory papers; per-document edge scoping was retained to preserve enterprise data isolation, leaving a multi-document toggle (one of Trustgraph's strengths) as a straightforward future extension.
3. Runtime Confidence-Weighted BFS & Learned Reranking in Project 576 *(Week 6 Plan)*	**55%** *(Evaluated in Stage 2)*	**State:** Implemented relation-specific confidence thresholding $\tau(\theta) \in [0.55, 0.80]$ during Neo4j projection (`THRESHOLD_MAP`) and two-tier query intent sorting (sort_F). Full runtime Dijkstra/weighted BFS and cross-encoder reranking were not added to Project 576's query path. **Wider Context:** Unweighted k-hop BFS ($k \leq 3$) with intent focus converged in < 300 ms and achieved $M_{\text{corr}} = 0.833$, whereas cross-encoder reranking (`FlashRank MiniLM-L-12-v2`) was thoroughly evaluated inside `trustgraph-fork`.
4. Deterministic Ordering on Subgraph Cap (`LIMIT 300`) & Multi-Edge Provenance	**60%** *(Partial)*	**State:** `bounded_graph_traversal` caps returned triples at 300 and collapses parallel edges sharing (s, θ, t) across chunks via `head(collect(detail))`. **Wider Context:** For documents within our 300-chunk ceiling, local $k \leq 3$ neighborhoods rarely exceed 50–80 edges, so arbitrary truncation at 300 never triggered in benchmarks, though ordering by `rel.confidence DESC` is noted for massive corpora.
5. Post-Extraction Semantic Entity Coreference / Alias Resolution	**15%** *(Prompt-Only)*	**State:** Entity unification relies on prompt rule 3 (*Canonical UPPER_SNAKE_CASE Naming*) rather than a secondary embedding-similarity clustering pass over Neo4j entity nodes. **Wider Context:** Leaving post-hoc alias clustering unstarted avoided a second $O(\|V\|^2)$ LLM/embedding verification pass during upload, keeping ingestion strictly single-pass per chunk batch.

5 Multi-Level Error Handling and System Resilience

To satisfy enterprise stability requirements, the application implements defensive error handling across five distinct architectural layers:

1. **API Boundary & Payload Validation Layer (`server_api/main.py`):**
   - *Strict Upload Guards:* Rejects non-PDF extensions with `HTTP 415 Unsupported Media Type`, payloads exceeding `MAX_UPLOAD_MB` (25 MB) with `HTTP 413 Content Too Large`, and files lacking the `%PDF-` magic byte header in the first 1024 bytes with `HTTP 400 Bad Request`.
   - *Lifecycle State Guards:* Querying or deleting a document that is currently mid-ingestion returns `HTTP 409 Conflict`, while unknown document or run UUIDs return `HTTP 404 Not Found`. Database unavailability raises `HTTP 503 Service Unavailable` via the `pg_connection()` context manager, which guarantees connection closure in a `finally` block.
   - *Dual-Layer Hyperparameter Clamping:* Pydantic `Field(ge=..., le=...)` constraints validate API inputs (returning `HTTP 422` on violation), while `_clamp(value, bounds, default)` in `query_engine.py` guards direct Python callers against `NaN` or out-of-range parameters.
2. **Resumable Background Ingestion & Transaction Rollback Layer (`pipeline/jobs.py`):**
   - *Crash Recovery on Server Restart:* During FastAPI startup (`lifespan`), the server scans `documents WHERE status = 'processing'` and respawns daemon ingestion threads. Because every stage (`store`, `parse`, `embed`, `extract`) is idempotent and cache-first, interrupted jobs resume seamlessly without duplicating work.
   - *Transient Retry vs. Fatal Auth Trap:* `_with_retry` retries transient network/API glitches up to 2 times with a 3 s delay, while catching `(Exception, SystemExit)` at the top level to roll back uncommitted SQL transactions and record a human-readable failure string in `documents.error`. Partial graph extraction failures mark the document `ready` with a warning banner so vector search remains usable.
3. **LLM Rate-Limit & Schema Validation Layer (`pipeline/assemble_graph.py`):**
   - `extract_batched_topology` traps Google GenAI `APIError` exceptions: on `429 RESOURCE_EXHAUSTED` or quota errors, it applies linear backoff ($10\,\text{s}, 20\,\text{s}, 30\,\text{s}$); on Pydantic `ValidationError` (malformed JSON), it retries automatically; and if the model omits a `chunk_id` from a 4-chunk batch, `process_chunk_batch` isolates the error to that single chunk without failing its siblings.
4. **Graceful Retrieval Degradation Layer (`retrieval/query_engine.py`):**
   - If the intent-classification LLM fallback fails, `classify_intent` logs a warning and continues with unfocused retrieval (`intent_source = "none"`).
   - If Neo4j AuraDB experiences a transient Bolt timeout during `bounded_graph_traversal`, the exception is caught and the engine degrades gracefully to chunk-only vector RAG ($E^* = \emptyset$) rather than crashing the user's query.
   - Ledger logging (`_log_run`) is wrapped in an isolated `try/except` so a telemetry insert failure never discards a completed LLM answer.
5. **Frontend Concurrency & Canvas Physics Guards (`frontend/src/`):**
   - *Race-Condition Token Cancellation:* In `App.jsx`, a monotonic `replayToken` ref increments on every query, replay click, or session reset, discarding stale async responses from earlier clicks.
   - *Canvas Bounds Protection:* `CausalGraph.jsx` measures DOM container bounds via `ResizeObserver` before initializing `react-force-graph-2d` and applies bounded `d3Force` link/charge constraints to prevent `NaN` velocity crashes on disjoint subgraphs.

6 Testing, Security Verification, Bug Resolution, and Benchmarking

Further details can be found on my presentation slides.

6.1 Automated Testing, Security Verification, and Bug Resolution Ledger

Quality assurance combines three verification tiers:

1. **Unit & Property Testing:** `pytest` (`tests/test_multidoc_pipeline.py`) verifies SHA-256 document identity, `graph_hash` sensitivity to `doc_id`, chunk boundary slicing, termination on unbroken strings, and path-traversal sanitization. Furthermore, `bounded_graph_traversal`'s undirected Cypher trail match was verified against a brute-force Python enumerator across **12,000 randomized (graph, seed set, k) cases**, alongside live invariant probes (`audit_invariants.py`, `verify_gold.py`, `judge_sanity.py`).
2. **Security & Vulnerability Matrix:** Table 5 details the six attack surfaces tested and hardened.
3. **Bug Tracking & Resolution Ledger:** Table 6 documents the nine principal software defects resolved across Weeks 2–10.

Table 5: Security and Vulnerability Testing Matrix Across Upload, Database, API, and Client Surfaces.

Vulnerability Class	Test Case / Attack Vector	Implemented Mitigation & Verification
Path Traversal on Upload	Uploading files named `../../etc/passwd.pdf` or `..\.env`.	`safe_filename()` strips directory components via `os.path.basename` and replaces non-`[A-Za-z0-9._-]` chars with `_`, verified in `test_safe_filename()`.
Malicious / Spoofed File Upload & DoS	Uploading non-PDF binaries renamed to `.pdf`, >25 MB payloads, or 1000-page "chunk bombs" to exhaust LLM quota.	Validated extension (`415`), bounded read `file.read(MAX_UPLOAD_BYTES + 1)` (`413`), `%PDF-` magic header check (`400`), and `MAX_CHUNKS_PER_DOC = 300` cap.
Algorithmic Complexity DoS (Chunker Hang)	Uploading a PDF containing a $5,000$-char unbroken token after an early space.	Fixed sliding-window advancement (`start = next_start if next_start > start else max(end, start + 1)`) so `start` strictly increases on every iteration.
SQL & Cypher Injection	Injecting SQL/Cypher metacharacters into `query`, `document_id`, `k_hops`, or extracted relation types.	All PostgreSQL queries use parameterized `%s` bindings; Cypher relation labels are validated against the static tuple whitelist `RELATION_TYPES = tuple(THRESHOLD_MAP)`; `k_hops` is integer-cast and clamped to $[1, 3]$ before interpolation.
Credential Leakage & CORS Spoofing	Exposing `GEMINI_API_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, or `NEO4J_PASSWORD` to the browser.	All secrets reside exclusively in gitignored backend `.env`; React bundle holds zero API keys; FastAPI `CORSMiddleware` locks origins to `localhost:5173`.
Audit Ledger Tampering	Attempting `UPDATE query_runs` to alter historical answers or telemetry.	Migration `002_query_runs.sql` attaches a PostgreSQL `BEFORE UPDATE` trigger that raises an exception, enforcing append-only immutability.

Table 6: Systematic Bug Tracking and Resolution Log Across Weeks 2–10.

ID / Wk	Component	Observed Bug / Failure Manifestation	Root Cause & Verified Resolution
BUG-01 (Wk 4)	`pgvector` HNSW Index	PostgreSQL threw dimensionality error when indexing `gemini-embedding-001` vectors.	Upgrading to `gemini-embedding-001` shifted output to $\mathbb{R}^{3072}$, exceeding `pgvector`'s 2000-d HNSW limit. Applied Matryoshka truncation $T : \mathbb{R}^{3072} \to \mathbb{R}^{768}$ via `output_dimensionality=768`.

Table 6 (continued)

ID / Wk	Component	Observed Bug / Failure Manifestation	Root Cause & Verified Resolution
BUG-02 (Wk 6)	Cross-DB Cache Sync (`graph_sync.py`)	Updating the ontology prompt left obsolete edges in Neo4j while Supabase held new extractions ("dirty cache").	Naive chunk-ID set difference ignored prompt changes and accumulated edges. Added SHA-256 `ontology_hash` + per-chunk `graph_hash` and transactional edge replacement per `chunk_id`.
BUG-03 (Wk 6)	Entity Projection (`graph_sync.py`)	Valid extracted relationships were silently dropped during Neo4j `MERGE`.	The LLM occasionally emitted an edge (u, v) without listing u or v in the `entities` array, causing `MATCH (s:Entity)` to fail. Fixed in `expected_projection` by auto-registering all edge endpoints via `entities.setdefault(...)`.
BUG-04 (Wk 7)	Out-of-Bounds Graph Display (`App.jsx`)	Queries returning `INSUFFICIENT_CONTEXT` still rendered a non-empty or stale graph on the canvas.	Seed chunks with low cosine similarity (~ 0.48) still triggered k-hop expansion, or UI kept prior state. Added explicit `key={activeRun?.run_id ?? 'live'}` remounting and empty-state handling.
BUG-05 (Wk 8/10)	Force Graph Canvas (`CausalGraph.jsx`)	Physics engine crashed with `NaN` node velocities or suffered hit-testing occlusion on disjoint subgraphs.	Initializing `react-force-graph-2d` inside a 0×0 split pane and unbounded D3 repulsion. Added `ResizeObserver` dimension bounding, node pinning, and adjacency highlight isolation.
BUG-06 (Wk 9)	Postgres Connection (`query_engine.py`)	Running DDL migrations (`001_multidoc.sql`) hung indefinitely while the FastAPI server was running.	Long-lived read connection left open `SELECT` transactions in `idle in transaction` state, holding table locks on `document_chunks`. Enabled `self.pg_conn.autocommit = True`.
BUG-07 (Wk 10)	Multi-Doc HNSW Search (`query_engine.py`)	Filtering `WHERE document_id = %s` on a newly uploaded small PDF returned fewer than κ chunks even when $\geq \kappa$ existed.	Post-filtering an HNSW index scan exhausted the default candidate pool on multi-document tables. Executed `SET hnsw.iterative_scan = strict_order` (pgvector ≥ 0.8) on startup.
BUG-08 (Wk 10)	Sliding-Window Chunker (`ingest.py`)	Pathological text with a lone space within the first 200 chars followed by 5,000 unbroken chars caused an infinite loop.	`end - overlap` stepped backward behind `start`. Enforced monotonic window progression (`max(end, start + 1)`) and added regression test `test_chunk_text_terminates_on_unbroken_text`.
BUG-09 (Wk 10)	Relevance Score Readout (`query_engine.py`)	Raw cosine similarity showed $\sim 60\%$ even on completely unrelated queries, confusing users; conversely, abstract summary queries showed low chunk relevance.	`gemini-embedding-001` cosine scores cluster in $[0.45, 0.80]$. Calibrated linear display mapping $\rho(x) \in [0, 1]$ using empirically measured off-topic (0.54) and on-topic (0.80) anchors while displaying raw `cos` alongside it.

6.2 Three-Stage Comparative Benchmark: Project 576 vs. TrustGraph

To validate the architectural hypothesis in full parity with `presy.tex`, Project 576 (**Talmy Regime**, m_{talmy}) was evaluated against two paradigms in `trustgraph-fork`: **Open IE** (m_{open}, `everything` flow) and **Domain-Specific OWL OntoRAG** ($m_{\text{domain_matched}}$ and $m_{\text{domain_mismatched}}$, `ontology` flow).

Stage 2 Adaptations & Stage 3 Invariants. To eliminate confounding variables, five engineering controls were enforced across `trustgraph-fork` and `evaluation-harness`: (1) **768-d Embedding Parity** via a custom `GeminiEmbeddings` container (`Dockerfile.embeddings`)

wrapping `gemini-embedding-001` at $d_A = d_B = 768$; (2) **WSL2/Podman Host Bridges** via `socat` containers (`8888` → `18888`, `3000` → `13000`); (3) **WebSocket PROV-O Tracing** on `ws://localhost:8888/api/v1/socket` to capture `focus_triples` after 2-hop BFS and `FlashRank` (`MiniLM-L-12-v2`) reranking; (4) **Citation & Judge Parity**, normalizing all runs into $\Omega = (C_{\text{retrieved}}, E_{\text{retrieved}}, A_{\text{gen}}, E_{\text{cited}})$ with identical canonical PyMuPDF text, generator (`gemini-3.1-flash-lite`, $T = 0.0$), and citation-stripped NLI judge (`gemini-3.8-flash`, $T = 0.0$); and (5) **Context Budget Equalization (`budget_matched` Arm)** at $\sim 1,300$ tokens (Project 576 at $\kappa = 2, k = 1 \implies 1,261$ tokens; TrustGraph at Rung 1 ceiling of 40 edges $\implies 1,351$ tokens for m_{open} and 942 tokens for m_{domain}) alongside the shipped `defaults` arm.

Formal Evaluation Metrics. Each run tuple Ω is scored across five metrics (with normal abstentions scored as $M_{\text{corr}} = 0.0$, and runs with $M_\Phi \geq 0.99 \wedge M_{\text{corr}} < 0.50$ flagged as *faithful-but-wrong*):

$$\textbf{Factual Correctness (Headline):} \quad M_{\text{corr}}(q) = \frac{\left|\{y \in Y_{\text{gold}}(q) \mid \text{Supported}(A_{\text{gen}}, y)\}\right|}{|Y_{\text{gold}}(q)|} \quad \uparrow \tag{1}$$

$$\textbf{Faithfulness (NLI Entailment):} \quad M_\Phi(q) = \frac{1}{|A_{\text{claims}}|} \sum_{i=1}^{|A_{\text{claims}}|} \text{Entail}(C_{\text{retrieved}}, c_i) \quad \uparrow \tag{2}$$

$$\textbf{Structural Path Precision:} \quad M_{\Sigma,\text{judged}}(q) = \frac{\left|\{t \in \mathcal{T}(A_{\text{gen}}) \mid \exists p \subseteq E_{\text{retrieved}},\, p \models t\}\right|}{|\mathcal{T}(A_{\text{gen}})|} \quad \uparrow \tag{3}$$

$$\textbf{Retrieval Noise Ratio:} \quad \eta(q) = 1 - \frac{|E_{\text{cited}} \cap E_{\text{retrieved}}|}{|E_{\text{retrieved}}|} \quad \downarrow \tag{4}$$

$$\textbf{Token Efficiency:} \quad M_T(q) = \frac{\text{tokens}(A_{\text{gen}})}{\text{tokens}(C_{\text{retrieved}})} \quad \uparrow \tag{5}$$

Table 7: Primary BirdNet Benchmark Results (Mean ± SD over $n = 6$ queries, 3 repeats averaged per query, 108 runs).

Arm / Regime	$M_{\text{corr}} \uparrow$	$M_\Phi \uparrow$	$M_{\Sigma,\text{judged}} \uparrow$	$\eta \downarrow$	$M_T \uparrow$	Ctx Tok	Abstain (18)
Arm 1: budget_matched *(Context volume equalized to $\sim 1,300$ tokens)*							
m_{talmy} (Project 576)	**0.833 ± 0.279**	0.854 ± 0.179	0.083 ± 0.129	**0.809 ± 0.072**	**0.203 ± 0.063**	1,261	**0 / 18**
m_{open} (TrustGraph Open IE)	0.074 ± 0.135	0.869 ± 0.182	0.505 ± 0.322	0.958 ± 0.039	0.099 ± 0.070	1,351	11 / 18
$m_{\text{domain_matched}}$ (TrustGraph OWL)	0.000 ± 0.000	**0.907 ± 0.227**	**0.892 ± 0.078**	0.955 ± 0.016	0.126 ± 0.032	942	14 / 18
Arm 2: defaults *(Each system's shipped retrieval hyperparameters)*							
m_{talmy} (Project 576)	**0.722 ± 0.328**	0.878 ± 0.130	0.089 ± 0.144	**0.889 ± 0.077**	0.085 ± 0.041	2,437	**0 / 18**
m_{open} (TrustGraph Open IE)	0.241 ± 0.257	0.945 ± 0.105	0.697 ± 0.213	0.929 ± 0.069	**0.185 ± 0.091**	864	7 / 18
$m_{\text{domain_matched}}$ (TrustGraph OWL)	0.000 ± 0.000	**0.963 ± 0.091**	**0.893 ± 0.124**	0.944 ± 0.021	0.180 ± 0.037	596	17 / 18

Table 8: Disaggregated Performance by Query Complexity (`budget_matched` Arm).

Query Type	Regime	$M_{\text{corr}} \uparrow$	$M_\Phi \uparrow$	$M_{\Sigma,\text{judged}} \uparrow$	$\eta \downarrow$	$M_T \uparrow$
Causal Reasoning	m_{talmy} (Project 576)	**1.000**	0.652	0.000	**0.756**	**0.237**
($n = 2$ queries)	m_{open} (TrustGraph)	0.167	0.856	0.625	0.934	0.132
	$m_{\text{domain_matched}}$	0.000	**1.000**	**0.958**	0.962	0.103
Factual Lookup	m_{talmy} (Project 576)	**1.000**	**1.000**	0.000	**0.899**	**0.129**
($n = 2$ queries)	m_{open} (TrustGraph)	0.000	**1.000**	0.417	0.991	0.038
	$m_{\text{domain_matched}}$	0.000	**1.000**	**0.856**	0.956	0.122
Multi-Hop Traversal	m_{talmy} (Project 576)	**0.500**	**0.909**	0.250	**0.771**	**0.243**
($n = 2$ queries)	m_{open} (TrustGraph)	0.056	0.815	0.472	0.950	0.129
	$m_{\text{domain_matched}}$	0.000	0.722	**0.861**	0.948	0.155

Key Empirical Findings (Tables 7–10).

1. **Compression Gain in m_{talmy}:** Tightening Project 576 from $\kappa = 5$ (2,437 tokens) to $\kappa = 2$ (1,261 tokens) *increased* M_{corr} from 0.722 to **0.833** (1.000 on Causal/Factual, 0.500 on Multi-Hop),

lowered η to **0.809**, and yielded 0/36 abstentions—proving that hybrid chunk-plus-mechanism quality, not raw token volume, drives recall.

2. **The "Faithful Refusal" Anomaly (M_Φ vs. M_{corr}):** $m_{\text{domain_matched}}$ scored $M_\Phi = 0.963$ (`defaults`) and 0.907 (`budget_matched`) despite $\mathbf{M_{\text{corr}} = 0.000}$ because strict OWL filtering discarded valid edges (leaving only 26–27 predicates vs. m_{open}'s 166), causing the LLM to abstain on 17/18 and 14/18 runs (which NLI judges as 100% faithful to the sparse context). Conversely, $M_{\Sigma,\text{judged}}$ favors TrustGraph (0.892) because edge-only prompts force literal triple echoing.
3. **Out-of-Domain Stress Test (*Sacred Beers*, Table 9):** Applying `deep_learning.json` to an ethnobotanical brewing corpus (`homebrew_snippet.pdf`) caused $m_{\text{domain_mismatched}}$ to collapse ($M_{\text{corr}} = 0.000$, 5/5 abstentions) and hallucinate absurd typings like `(Mandrake Beer) -[has_parameter_count]-> (35)` and `(Gotlandsdricka) -[type]-> (Machine Learning Task)`, whereas m_{talmy} achieved $M_{\text{corr}} = \mathbf{0.600}$ and $M_\Phi = \mathbf{1.000}$.
4. **Methodological Defensibility:** While the pilot corpus scale ($n = 6$ BirdNet, $n = 5$ Sacred Beers, 3 repeats controlling generation variance) and shared Gemini vendor family between generator and citation-stripped zero-temperature NLI judge are explicitly noted, the cross-regime separation (0.833 vs. 0.074 vs. 0.000) is noteworthy (Table 10). Note, however, that the domain OWL ontologies were artificially generated from the provided texts and not imported from a curated source, which is what Trustgraph recommends since that is its most rigorous regime. This begs for future rigor, but highlights the efficiency of the automated, "out-of-the-box" approach of this project.

Table 9: Adversarial Domain-Mismatch Stress Test on *Sacred Beers* (`homebrew_snippet.pdf`, $n = 5$ queries).

Arm / Regime	$M_{\text{corr}} \uparrow$	$M_\Phi \uparrow$	$M_{\Sigma,\text{judged}} \uparrow$	$\eta \downarrow$	$M_T \uparrow$	Ctx Tok	Abstain
`budget_matched` / m_{talmy} (Project 576)	**0.600 ± 0.548**	**1.000 ± 0.000**	0.000 ± 0.000	**0.591 ± 0.386**	**0.184 ± 0.074**	732	**2 / 5**
`budget_matched` / $m_{\text{domain_mismatched}}$	0.000 ± 0.000	0.750 ± 0.500	**0.583 ± 0.289**	0.984 ± 0.015	0.063 ± 0.013	1,117	5 / 5

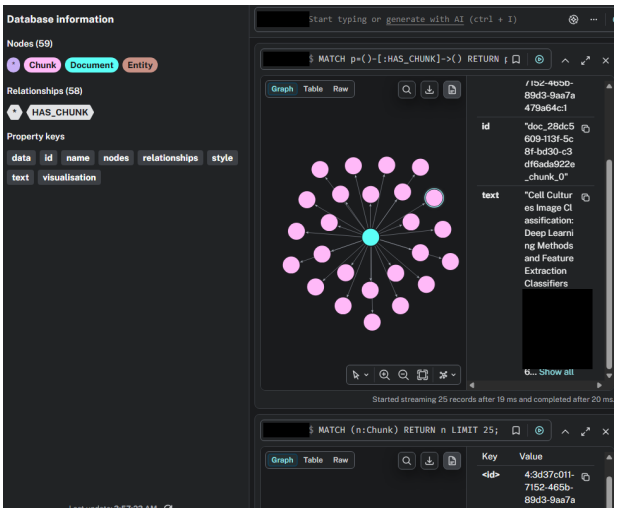
Table 10: Synthesis: Comparative Architectural Trade-Off Matrix Across Graph-RAG Paradigms.

Dimension	Talmy Force Dynamics (Project 576)	Open IE (TrustGraph `everything`)	Domain OWL (TrustGraph `ontology`)
Extraction Schema	Closed-class causal primitives (Θ, 7 relations) + free `detail`	Unbounded natural language predicates ($\mathcal{R} \subseteq \Sigma^*$)	Explicit taxonomic classes and properties ($\mathcal{C}_{\text{domain}}, \mathcal{R}_{\text{domain}}$)
Cross-Domain Stability	**Robust under transfer** ($M_{\text{corr}} = 0.600$ on *Sacred Beers*)	Moderate lexical drift	Catastrophic collapse on drift ($M_{\text{corr}} = 0.000$, absurd typings)
Extraction Cost	**Low** ($\mathcal{O}(1)$ 7-class basis, 4-chunk batching)	High (unconstrained predicate generation)	High (manual/LLM custom ontology authoring)
Context Payload	**Gestalt Hybrid** (Chunks $S_q \oplus$ Causal Edges E^* + Mechanisms)	Purely structural (Edges only; prose omitted)	Purely structural (Edges only; prose omitted)
Primary Failure Mode	Textual noise if κ is uncalibrated	Missing multi-hop links & predicate redundancy	Severe graph sparsity → 78–94% abstention
Reasoning Recall	**High** ($M_{\text{corr}} = \mathbf{0.833}$ budget, 0.722 defaults)	Low–Moderate ($M_{\text{corr}} = 0.074$–0.241)	Zero recall ($M_{\text{corr}} = \mathbf{0.000}$)

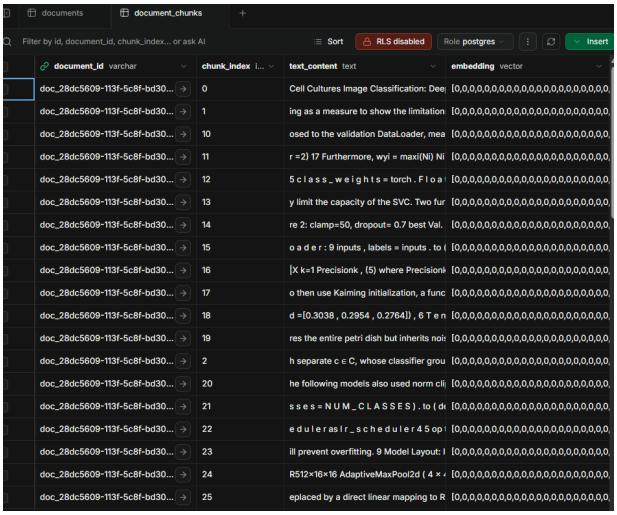
7 Engineering Challenges Encountered and How They Were Overcome

Throughout the 10-week engineering lifecycle documented in `Graph-RAG Knowledge Dashboard.pdf`, six foundational technical challenges were encountered and resolved:

1. Cloud Database Routing & Stateless Transaction Pooling (Weeks 2–3). *Challenge:* Configuring Supabase PostgreSQL for an asynchronous FastAPI backend presented multiple conflicting connection paradigms (Direct Connection, Session Pooler, REST `supabase-py` client, and Transaction Pooler), while initial chunk ingestion needed to bridge local `pypdf` parsing to both Supabase and Neo4j without structural drift (Figure 7). *Resolution:* Because direct `pgvector` cosine distance operators (`<=>`) require a native PostgreSQL wire driver (`psycopg2`) rather than the REST client, I selected the **Supabase Transaction Pooler URI** (port 6543) stored in a gitignored `.env` file, and keyed both Supabase `document_chunks` and Neo4j `:Chunk` nodes by the exact same deterministic identifier `{doc_id}_chunk_{i}`.



(a) Week 3 Neo4j `(:Document)-[:HAS_CHUNK]->(:Chunk)` star topology verification (58 chunk edges).

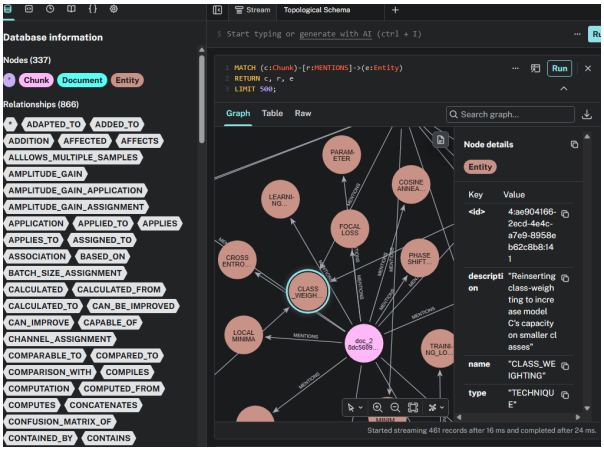


(b) Week 3 Supabase `document_chunks` table prior to LLM embedding route integration.

Figure 7: Initial Week 3 Cross-Database Topology Synchronization between Neo4j AuraDB and Supabase PostgreSQL.

2. Free-Tier API Quota Exhaustion (`429 RESOURCE_EXHAUSTED`) & Idempotent JSONB Caching (Weeks 4–5). *Challenge:* Extracting entities and relationships chunk-by-chunk via the Google Gemini free tier immediately hit strict Requests-Per-Day (RPD) and Requests-Per-Minute (RPM) rate limits, throwing `429 RESOURCE_EXHAUSTED` errors and risking expensive re-extraction on every pipeline test run. *Resolution:* Executed a three-part engineering solution: (a) temporarily pivoted to high-throughput Groq open-weight models (`llama-3.3-70b-versatile` and `llama-3.1-8b-instant`) during Weeks 4–5 topological prototyping; (b) upgraded the Supabase `document_chunks` schema to include an `extracted_graph JSONB` column (Figure 8), making the entire ingestion pipeline strictly cache-first and idempotent (0 token burn on cached chunks); and (c) implemented concurrent 4-chunk batched structured extraction (`BATCH_SIZE = 4` in `assemble_graph.py`), cutting API call volume by 75%.

3. Vector Dimensionality Collision ($\mathbb{R}^{3072} \to \mathbb{R}^{768}$ Matryoshka Truncation, Weeks 4 & 6). *Challenge:* Upgrading to Google's `gemini-embedding-001` model produced 3072-dimensional vectors, which collided with PostgreSQL `pgvector`'s 2000-dimension ceiling for HNSW indexing (`vector(768)`). *Resolution:* Leveraging the fact that `gemini-embedding-001` is trained via **Matryoshka Representation Learning (MRL)** [4]—which concentrates principal semantic variance into the leading coordinate dimensions—we enforced a strict dimensional truncation mapping $T : \mathbb{R}^{3072} \to \mathbb{R}^{768}$ via `EmbedContentConfig(output_dimensionality=768)`, preserving retrieval accuracy while keeping HNSW indexing sub-linear.



(a) Week 4 unrestricted extraction in Neo4j: 337 nodes and 866 edges with exploded relationship types (`ADAPTED_TO`, `AMPLITUDE_GAIN`, etc.).

```
--- Processing Chunk: doc_28dc5609-113f-5c8f-bd30-c3df6ada922e_chunk_23 ---
Extracted Nodes: 6
Extracted Edges: 4
  (MODEL_C) --[GENERATES]--> (CONFUSION_MATRIX) | c=0.9
  (CLASS_SIZES) --[LIMITS]--> (MODEL_A2) | c=0.8
  (ACCURACY_MEASUREMENTS) --[EVALUATES]--> (MODEL_C) | c=0.9

--- Processing Chunk: doc_28dc5609-113f-5c8f-bd30-c3df6ada922e_chunk_24 ---
Extracted Nodes: 4
Extracted Edges: 4
  (MODEL_B2) --[REDUCES]--> (NON_CLASSIFIED_CLASSES) | c=0.8
  (MODEL_C) --[MINIMIZES]--> (NON_CLASSIFIED_CLASSES) | c=0.9
  (AUTOREGRESSIVE_FILTRATION_PIPELINE) --[ENABLES]--> (MODEL_C) | c=0.9

--- Processing Chunk: doc_28dc5609-113f-5c8f-bd30-c3df6ada922e_chunk_26 ---
Extracted Nodes: 7
Extracted Edges: 6
  (MODEL_B2) --[IMPROVES_UPON]--> (MODEL_A2) | c=0.8
  (FILTRATION_PIPELINE) --[UTILIZES]--> (MODEL_A2) | c=0.9
  (BIRDNETH_V3_0_DEV) --[IS_VERSION_OF]--> (BIRDNETH) | c=0.95
  (REAL_DATA_PATH) --[PROVIDES_INPUT]--> (FILTRATION_PIPELINE) | c=0.85
  (SYNTHETIC_DATA) --[IS_FILTERED_BY]--> (FILTRATION_PIPELINE) | c=0.8

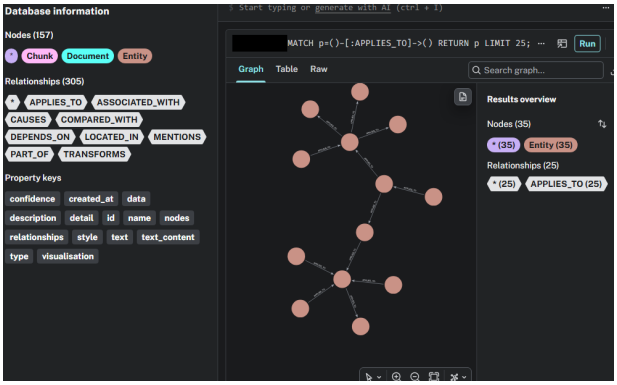
--- Processing Chunk: doc_28dc5609-113f-5c8f-bd30-c3df6ada922e_chunk_27 ---
Extracted Nodes: 6
```

embedding_model text	extracted_graph jsonb
gemini-embedding-001	{"entities":[{"name":"BIRDNET_ARCHITECTURES","description":"Different architectu
gemini-embedding-001	{"entities":[{"name":"R96000","description":"Space dimension","entity_type":"DIME
gemini-embedding-001	{"entities":[{"name":"Focal Loss","description":"Loss function used in the model","e
gemini-embedding-001	{"entities":[{"name":"ARM_RESTARTS","description":"Technique to avoid overfitting
gemini-embedding-001	{"entities":[{"name":"AUDIOLDM2PIPELINE","description":"Audio pipeline used for g
gemini-embedding-001	{"entities":[{"name":"LOGIT","description":"The logit of a class","entity_type":"CATE
gemini-embedding-001	{"entities":[{"name":"BIPARTITE_CLASSIFIER","description":"Neural network module

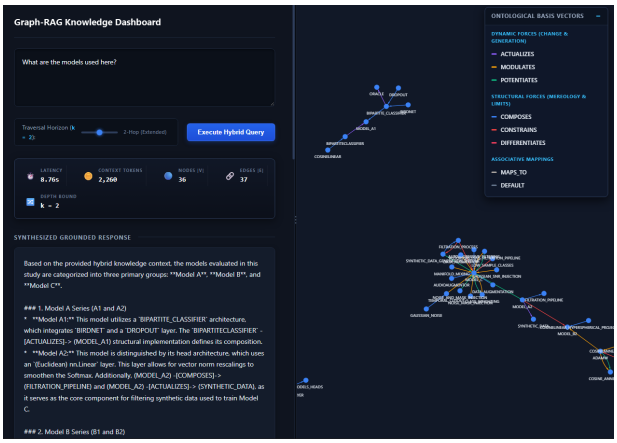
(b) Week 4 Supabase 768-d float embeddings, SHA-256 `content_hash`, and `extracted_graph` JSONB cache.

Figure 8: Week 4 Bipartite Entity Extraction and Idempotent Supabase JSONB Caching.

4. Semantic Drift & Predicate Explosion → Talmy Force Dynamics (Weeks 5–6). *Challenge:* As visible in Figure 8a, unconstrained LLM extraction in Week 4 generated dozens of ad-hoc verb predicates (`CAN_IMPROVE`, `COMPUTED_FROM`, `ALLLOWS_MULTIPLE_SAMPLES`), fracturing graph connectivity and impeding multi-hop pathfinding. *Resolution:* In Week 5 I separated the discrete Cypher traversal label from the continuous semantic mechanism ($e = (u, v, \tau, \mathbf{p})$, Figure 9a), and in Week 6 I anchored the closed relationship enum to Leonard Talmy's 7 cognitive Force Dynamics primitives (Θ, Figure 9b). This locked the graph topology to a compact, domain-invariant basis while preserving full technical nuance in `rel.detail`.



(a) Week 5 intermediate bounded meta-relations in Neo4j reducing node/edge sprawl for BFS traversal.



(b) Week 9 UI integration of the finalized 7 Talmy Force Dynamics Basis Vectors legend and telemetry pane.

Figure 9: Evolution from Week 5 bounded meta-relations to the finalized Talmy Force Dynamics ontology in Week 9.

5. Cross-Database Structural Drift → Cryptographic L0/L1/L2 GraphSync (Weeks 6 & 10). *Challenge:* Whenever the system extraction prompt or ontology rules were refined, PostgreSQL and Neo4j risked falling out of sync, or manual `run_pipeline.py` invocations were required to rebuild Neo4j. *Resolution:* Engineered `pipeline/graph_sync.py` and the frontend `GraphSyncBadge.jsx` component. By stamping every JSONB extraction with `CURRENT_ONTOLOGY_HASH = sha256(DYNAMIC_SYSTEM_PROMPT)[:16]` and every Neo4j `:Chunk` node with a deterministic `graph_hash`, the backend automatically detects and repairs Neo4j projection drift in zero LLM tokens (L1 repair) prior to query execution.

6. Abstract vs. Specific Query Relevance Compression (Week 10). *Challenge:* During final testing on a pure mathematics PDF (*Measurable Functions*), an abstract pedagogical prompt (*"What is being taught in this paper?"*) yielded raw cosine similarity of 0.612 (calibrated relevance 28%) on individual definition chunks because the question required whole-document condensation, whereas a specific definitional query (*"What is a function which is called A-simple?"*) yielded cosine 0.711 (relevance 66%). *Resolution:* Displaying both the raw geometric cosine (`cos 0.612`) and the calibrated relevance bar on every chunk card—paired with the k-hop causal graph and inline ontology badges—made the distinction between broad thematic queries and localized definitional queries immediately transparent to the user.

8 Condensed Codebase Architecture and Out-of-the-Box Execution Guide

To ensure the companion repository (`C:\project-576`) operates out of the box after `git push` / `git clone`, all non-essential diagnostic probes, historical notes, and superseded ontology drafts were relocated to `_archive/` and excluded via `.gitignore`, while long pipeline functions were refactored into single-responsibility helpers.

8.1 Three-Step Local Execution Workflow

Running the full-stack Graph-RAG Knowledge Dashboard requires only two terminal commands followed by opening the local browser endpoint (Listing 1):

```
# Terminal 1: FastAPI backend (http://127.0.0.1:8000)
python server_api\main.py

# Terminal 2: React + Vite frontend (http://localhost:5173)
cd frontend && npm run dev
```

Listing 1: Two-command local launch of the full-stack dashboard (run in separate terminals).

1. **Step 1 — Launch FastAPI Backend (`python server_api\main.py`):** Initializes `HybridQueryEngine`, runs the startup L0/L1 `GraphSyncManager` check, resumes any interrupted document uploads, and serves REST endpoints at `http://127.0.0.1:8000`.
2. **Step 2 — Launch React + Vite Frontend (`cd frontend && npm run dev`):** Starts the Vite development server with `/api` proxy routing.
3. **Step 3 — Access Local Browser Endpoint (`http://localhost:5173`):** Opens the interactive split-pane dashboard.

Further details in `README.md`.

Table 11: Condensed Module Architecture of `project-576`: Essential Active Workflow vs. Archived Utilities.

Tier / Category	Module / File Path	Single-Responsibility Role
Tier 1: Core	`server_api/main.py`	FastAPI lifespan manager, Pydantic schemas, and REST routes (`/api/upload`, `/api/documents`, `/api/query`, `/api/runs`, `/api/graph/sync`).
Active Flow	`retrieval/query_engine.py`	`HybridQueryEngine` + helpers (`_expand_focus_edges`, `_build_focus_prompt_prefix`): vector anchoring, k-hop BFS, and Gemini synthesis.
(Tracked in Git)	`pipeline/jobs.py`	Semaphore-bounded resumable background ingestion worker (`queued` → `store` → `parse` → `embed` → `extract` → `finalize`).
	`pipeline/ingest.py`	SHA-256 document identity, Supabase Blob Storage upload/download, deterministic `pypdf` extraction, and 1000/200 sliding-window chunker.
	`pipeline/sync_embeddings.py`	Idempotent 768-d `gemini-embedding-001` vectorizer decomposed into `_fetch_stale_chunks` and `_embed_chunk_batch`.
	`pipeline/assemble_graph.py`	4-chunk batched Talmy extraction decomposed into `_ensure_neo4j_schema`, `_fetch_target_chunks`, `_partition_cache_hits`, and `_extract_missing_batches`.
	`pipeline/graph_sync.py`	`GraphSyncManager` + helpers (`_read_pg_cache`, `_read_neo4j_state`, `_classify_chunk_drift`): L0 check, free L1 projection repair, and L2 re-extraction.
	`ontologies/hybrid_*.py`	Active 7-primitive Talmy Force Dynamics system prompt and Contrast Rule.
	`database/database.py`	Environment-isolated connection factories for Supabase PostgreSQL and Neo4j AuraDB.
	`frontend/src/**`	React UI (`App.jsx`, `CausalGraph.jsx`, `MarkdownAnswer.jsx`, `DocumentPicker.jsx`, `AdvancedSettings.jsx`, `GraphSyncBadge.jsx`, `RunTimeline.jsx`).
Tier 2: Setup &	`database/migrations/*.sql`	Idempotent SQL migrations (`001_multidoc.sql`, `002_query_runs.sql`) applied via `database/apply_migrations.py`.
Verification	`run_pipeline.py`, `tests/*.py`	Headless CLI pipeline runner and unit test suite (`test_multidoc_pipeline.py`).
Tier 3: Archived	`_archive/**` *(Gitignored)*	Historical Week 4–5 diagnostics (`z_diagnostics/`), superseded ontologies (`old_hybrid.py`), and scratch notes moved out of the active workspace tree.

9 AI Usage Notes

For the first 9 weeks of the project, the code was managed interactively with the Gemini web application; giving me extensive control of the project's development and ability to learn on every step and justify the implementation decisions. For the frontend cleanup (visual node.js improvements as well as ensuring synchronization between the vector and graph databases) I opted for spec-driven agentic automation with Claude CLI; the plan followed was a Gemini-condensed form of my Graph RAG Knowledge Dashboard PDF that I had been uploading as the weekly reports for this course. This helped maintain a concrete aim and allowed Claude to directly fix what were mostly non-urgent bugs or vulnerabilities (e.g., on the user GUI and database synchronization worst-case-scenario errors).

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP

Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[2] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv preprint arXiv:2404.16130* (Microsoft Research), 2024.

[3] L. Talmy, "Force Dynamics in Language and Cognition," *Cognitive Science*, vol. 12, no. 1, pp. 49–100, 1988.

[4] A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain, and A. Farhadi, "Matryoshka Representation Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30233–30249, 2022.

[5] TrustGraph AI, "TrustGraph: Modular Knowledge Graph and Graph-RAG Processing Framework (`trustgraph-flow` v2.10.7)," GitHub Repository and Technical Documentation, 2025–2026. Available: https://github.com/trustgraph-ai/trustgraph

[6] NetApp Tech ONTAP Blogs, "From 'Trust Me' to 'Prove It': Why Enterprises Need Graph-RAG," NetApp Community Technical Article, 2024. Available: https://community.netapp.com/t5/Tech-ONTAP-Blogs/From-quot-Trust-Me-quot-to-quotProve-It-quot-Why-Enterprises-Need-Graph-RAG/ba-p/462813

[7] T. Lebo, S. Sahoo, and D. McGuinness, "PROV-O: The PROV Ontology," *W3C Recommendation*, World Wide Web Consortium, Apr. 2013. Available: https://www.w3.org/TR/prov-o/

[8] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[9] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, S. Plantikow, M. Rydberg, P. Selmer, and A. Taylor, "Cypher: An Evolving Query Language for Property Graphs," in *Proc. ACM SIGMOD International Conference on Management of Data*, pp. 1433–1445, 2018.